

Abstract

1 Introduction

Bipartite Matching is the problem of finding the largest non-neighboring subset of edges of a bipartite graph. It is well-known and well-studied since it can be used to model a variety of real life problems since a bipartite graph can reflect a 'worker-can-fulfill-task' relation and the matching is an assignment of tasks. Online Bipartite Matching is a variant of this problem where the entire graph isn't initially visible and matching decisions have to be made 'on the fly' as the graph is revealed. This model is just as useful, if not more so, in real life scenarios since in the context of many services all tasks aren't known initially.

This simplified model, however, is often a lower bound, not a perfect solution since the assumptions of the model are harsher than reality. For this reason we examine the performance of various algorithms in slightly different models

The contents of the work will be as follows:

2. Preliminaries
3. The classical model introduced by Karp et al
4. The advice model, where the algorithm may receive imperfect advice
5. final degree model, where the degree of vertices in final graph is always known

2 Preliminaries

for matching M and vertex v matched in M , $M(v)$ is the vertex matched to v in M

3 Semi-Online Model

In this section we will introduce the classical smi-online model introduced by Karp et al [KVV90]

In this model the two anticliques of the bipartite graph are divided into offline and online agents. Offline agents are vertices that belong to one anticlique and are always visible. Online agents are the vertices from the other anticlique and arrive in the online fashion. When an online agent arrives edges incident to it are revealed.

The adversary is oblivious as an adaptive adversary is trivial.

Remark 3.1. *No algorithm has a competitiveness asymptotically higher than $\frac{1}{2}$ against an adaptive adversary*

Proof. Begin with $2n$ offline agents V and $k = 0, X_0 = \emptyset$.

Adversary repeats the following n times: reveals an agent u connected to $V \setminus X_k$. $k = k + 1$, for some unmatched $v \in V$; $V_{k+1} = V_k \cup \{v\}$.

Then adversary reveals n agents U' connected to already matched offline agents. The ultimate size of the matching is n .

In the final graph (figure), all matched vertices V' are connected to all online agents and all unmatched v_i agents are connected to all u_j such that $j \leq i$. Clearly, it is possible to match v_i to u_i and V' to U' , forming a matching of size $2n$

□

Remark 3.2. *In the semi-online model, any deterministic algorithm has a competitiveness no greater than $\frac{1}{2}$, there is a deterministic algorithm with competitiveness of $\frac{1}{2}$*

Algorithm 1 Naive

on Arrival of vertex v : match it with any offline agent if available

Proof. Trivially the competitiveness cannot be greater than $\frac{1}{2}$ as against a deterministic algorithm there is no difference between an adaptive and an oblivious adversary.

To prove that there is a deterministic algorithm with competitiveness of $\frac{1}{2}$ consider the a naive algorithm.

for every $(u, v) \in E(G)$ either u or v is matched. Therefore at least half of all vertices that could be matched are matched, therefore the competitiveness isn't lower than $\frac{1}{2}$ □

The main discovery of the Karp paper is the following algorithm

Algorithm 2 Ranking

on Initialization: create a random permutation of the offline agents σ

on Arrival of vertex v : match it with the first available offline neighbor according to σ

as well as the following theorems

Theorem 1. *Ranking has asymptotic competitiveness of $1 - \frac{1}{e}$*

Theorem 2. *No algorithm has asymptotic competitiveness higher than $1 - \frac{1}{e}$*

3.1 Ranking Competitiveness

In this section we present a simple proof for Ranking competitiveness (Theorem 1) by [BM08]

Lemma 3.1. *Competitiveness is determined by graphs with a perfect matching*

Proof. Let G be a bipartite graph without a perfect matching, with orderings of offline agents σ and of online agents π . Consider G' which is a copy of G without a vertex x and orderings π', σ' induced by $V \setminus \{x\}$.

Observe that $m = \text{Ranking}(G, \pi, \sigma)$ being the matching found by the Ranking algorithm on G with ranking π and arrival order σ is either identical to $m' = \text{Ranking}(G', \pi', \sigma')$ or has one more vertex pair. This is apparent since either:

1. x isn't in m ,
2. $x' = M(x)$ isn't matched in m' , or
3. x' is matched to y in m'

In the case of 3 there is an alternating path of edges in m' and those that are in m but not in m' . When this path ends in the same partition as x then m is larger than m' by 1

This also leads to the conclusion that if the respective matching sizes are different, they differ by a single alternating path.

Now consider the maximal matching of G and the effect the removal of the vertices outside of it will have on the competitiveness. The size of the maximal matching will (of course) not change, while the Ranking's solution decreases by at most 1 with each removal, implying that this process of reducing G to G_p with a perfect matching didn't increase the competitiveness, meaning that analysis of graphs without perfect matchings is redundant. $\square$

In the following section graphs are assumed to have perfect matchings. Let M^* be that perfect matching.

Lemma 3.2. *Let u online vertex, $v = m^*(u)$, σ permutation.*

If v isn't matched by $\text{Ranking}(\sigma)$ then u is matched by $\text{Ranking}(\sigma)$ to v' with a lower rank

Proof. Trivial since u, v are connected, therefore for v to remain unmatched u must've been matched to another vertex, which must've in turn had a lower rank than v $\square$

Lemma 3.3. *Let u online vertex, $v = m^*(u)$, σ permutation, σ_i permutation obtained by removing v and inserting it into sigma at the i -th place.*

If v isn't matched by $\text{Ranking}(\sigma)$ then for every i $\text{Ranking}(\sigma_i)$ matches u with a vertex v_i such that $\sigma_i(v_i) \leq \sigma(v) = t$

Proof. $m = \text{Ranking}(\sigma), m_i = \text{Ranking}(\sigma_i)$.

Symmetrically to Lemma 3.1 m, m_i are either identical or differ by a single alternating path. It is easy to see that offline vertices in that path are of increasing rank in σ_i . Therefore $v_i = m_i(u), v' = m(u), \sigma_i(v_i) \leq \sigma_i(v')$

Using Lemma 3.2 we get $\sigma(v') < t$ and since $|\sigma_i(v') - \sigma(v')| \leq 1$ we receive $\sigma_i(v_i) < 1 + t$ equivalent to $\sigma_i(v_i) \leq t$ on integers $\square$

The final lemma required to complete the proof uses this result directly.

Lemma 3.4. *The probability over σ that the offline vertex of rank t is matched by $\text{Ranking}(\sigma)$ denoted x_t fulfills the following: $1 - x_t \leq \frac{1}{n} \sum_{1 \leq s \leq t} x_s$*

Proof. Let σ be a random permutation of offline vertices. Define σ' with the following random process: choose an offline vertex randomly with equal probability, take it out of σ and insert it into the t -th rank. Let v be the vertex of rank t , $u = m^*(v)$, R_{t-1}

online vertices matched by the algorithm to offline vertices of rank $\leq t-1$ probability that v is not matched is $1 - x_t$. Expected cardinality of R_{t-1} is

$$\sum_{1 \leq s \leq t-1} x_s.$$

Applying Lemma 3.3 with $\sigma := \sigma'$, i chosen such that $\sigma'_i = \sigma$: if v isn't matched by $\text{Ranking}(\sigma')$ then u is matched by $\text{Ranking}(\sigma)$ to a vertex v' such that $\sigma(v) \leq t$ implying $u \in R_t$

Conditional on σ , $u \in R_t$ has a probability of $\frac{|R_t|}{n}$ since they are independent.

We now have two random events E, F that fulfill the condition that $E \implies F$.

It is obvious to see that $\mathbb{P}(E) \geq \mathbb{P}(F)$

The lemma follows □

With Lemma 3.4 the Theorem can be proved directly.

G has a perfect matching implies that competitiveness is $\inf \frac{1}{n} \sum_{1 \leq s \leq n} x_s$.

Let $S_t = \sum_{1 \leq s \leq t} x_s$. Lemma 3.4 can be rewritten as $S_t(1 + \frac{1}{n}) \geq 1 + S_{t-1}$ and

the competitiveness ratio as $\frac{1}{n} S_n$.

Infimum occurs therefore when all inequalities are tight, resulting in $S_t = \sum_{1 \leq s \leq t} (1 - \frac{1}{n+1})^s$

for all t , implying that competitiveness is at least $\frac{1}{n} \sum_{1 \leq s \leq n} (1 - \frac{1}{n+1})^s = 1 - (1 - \frac{1}{n+1})^n \xrightarrow{n \rightarrow \infty} 1 - \frac{1}{e}$

Thus the theorem is proven

3.2 Ranking Optimality

In this section we present a proof for Ranking Optimality (Theorem 2) by [KVV90]. The main vehicle for this is Yao's lemma with a construction of a class of matching instances

Algorithm 3 RGA

on Arrival of vertex v : match it with a random available offline neighbor

Definition 3.1. *bipartite online matching instance (n, σ) where σ is a permutation of $[n]$ is constructed as follows:*

- offline vertices are f_i for $i \in [n]$
- online vertices are n_j for $j \in [n]$
- f_i, n_j are connected iff $\sigma(i) \leq j$

The collection of all $n!$ (n, σ) instances is M . $U(M)$ is the uniform distribution over M . M^* is $(n, (1, \dots, n))$, the original instance without permutation [figure] notice that perfect matching is achieved only if f_i is matched with $n_{\sigma(i)}$ for all $i \in [n]$

Lemma 3.5. *Let DGA be a greedy deterministic algorithm. The expected performance of DGA on a random instance from $U(M)$ bounds the expected performance of RGA on M^**

$$\mathbb{E}_{m \sim U(M)} [|\text{DGA}(m)|] = \mathbb{E}[\text{RGA}(M^*)]$$

Proof. To prove the lemma we first need to prove two claims:

Claim 1 for DGA on (n, σ) as well as RGA on M^* if at time t there are k eligible offline vertices then they are equally likely to be any set of k vertices among the last $n - t + 1$ denominated $F(t)$

Claim 2 for each k the probability that there are k eligible offline vertices at time t , denominated p_t , is the same for DGA on (n, σ) as well as RGA on M^*

Both claims are proven with induction on the 'step' of the algorithm - the amount of online vertices that have already arrived. Note that induction base at step 0 is trivial since all offline vertices are eligible.

Note that a vertex is available if it can still be matched ie isn't matched and isn't unmatchable

Proof. of Claim 1
for RGA.

$$\Pr[S \text{ Available at time } t+1] = \sum_{v \notin S} \Pr[S \cup \{v\} \text{ Available at time } t] \Pr[v \text{ picked at time } t]$$

and since probabilities at time t are equal and the probability of choosing all v is either equal or 0 then the condition is maintained

for DGA, since it is deterministic and the algorithm can't read labels, then the subsequent matching is determined by the current available set, which is uniformly random, implying the next one is also uniformly random. $\square$

Proof. of Claim 2

From Claim 1 we know that, conditional on the amount of eligible vertices, their placement is universal $\square$

The lemma follows directly from Claim 2 since the size of the matching is equal to the amount of steps required for there to be no eligible vertices $\square$

By using Yao's lemma we receive the following: for any algorithm ALG

$$|\text{ALG}(M)| := \min_{m \in M} \mathbb{E}[|\text{ALG}, m|] \leq \max_{\text{DALG} m \sim U(M)} \mathbb{E}[|\text{DALG}(m)|] = \mathbb{E}[|\text{RGA}(M^*)|] \quad (1)$$

Since (as proven in the previous section) only instances with perfect matchings need be considered in competitiveness analysis, competitiveness of all algorithms is upper bounded by $\mathbb{E}[|\text{RGA}(M^*)|]/n$

Now, all that is required to prove the following lemma

Lemma 3.6. $\mathbb{E}[|\text{RGA}(M^*)|] \leq n(1 - \frac{1}{e}) + o(1)$

The rest of this section is the proof of this lemma.

Let $\text{rem}(t)$ be the random variable representing the number of remaining online vertices and $\text{avl}(t)$ be the number of offline vertices available for matching, at the time t . Δf is used conventionally, as the change of $f(t)$ over t .

Δrem is always -1 while Δavl is either -1 or -2 . It is only -2 if n_t is being matched and f_t is available (event A_t^1) and it isn't matched to n_t (event A_t^2) since f_t becomes unmatchable.

Note that rem is essentially a clock ticking backwards since it has value $n-t$

$\Pr[A_t^1 | \text{avl}, \text{rem}] = \frac{\text{avl}}{\text{rem}+1}$ since from Claim 1 from Lemma 3.5 we know that all avl sized subsets are equally likely to be the available subset

$\Pr[A_t^2 | \text{avl}] = 1 - \frac{1}{\text{avl}} = \frac{\text{avl}-1}{\text{avl}}$ since matched vertex is chosen randomly

Since both events are independent $\Pr[A_t^1 \wedge A_t^2] = \frac{\text{avl}-1}{\text{rem}+1}$

$\mathbb{E}[\Delta \text{avl}] = (-2) \frac{\text{avl}-1}{\text{rem}+1} + (-1)(1 - \frac{\text{avl}-1}{\text{rem}+1})$

finally

$$\mathbb{E}[\Delta \text{avl}] = -1 - \frac{\text{avl}(t) - 1}{\text{rem}(t)} \quad (2)$$

implying $\frac{\mathbb{E}[\Delta \text{avl}]}{\mathbb{E}[\Delta \text{rem}]} = 1 + \frac{\text{rem}(t)-1}{\text{avl}(t)}$ since Δrem is always -1

Though this is a discrete Markov Chain, we can approximate it by treating it as a continuous function, whose rate of growth is already known

$$\frac{d \text{avl}}{d \text{rem}} = 1 + \frac{\text{avl} - 1}{\text{rem}} \quad (3)$$

We also have the initial conditions, $\text{avl} = \text{rem} = n$, meaning we can find an approximating equation by solving Equation 3

$$\begin{aligned}
& \frac{d\text{avl}}{d\text{rem}} = 1 + \frac{\text{avl} - 1}{\text{rem}} \text{ can be rewritten as} \\
& \frac{d\text{avl}}{d\text{rem}} - \frac{\text{avl}}{\text{rem}} = \frac{\text{rem} - 1}{\text{rem}} \text{ multiply both sides by } m(\text{rem}) = e^{\int -1/\text{rem} d\text{rem}} \\
& \frac{\frac{d\text{avl}}{d\text{rem}}}{\text{rem}} + \frac{d}{d\text{rem}} \left(\frac{1}{\text{rem}} \right) \text{avl} = \frac{\text{rem} - 1}{\text{rem}^2} \text{ apply product rule to LHS} \\
& \frac{d}{d\text{rem}} \left(\frac{\text{avl}}{\text{rem}} \right) = \frac{\text{rem} - 1}{\text{rem}^2} \text{ integrate both sides} \\
& \int \frac{d}{d\text{rem}} \left(\frac{\text{avl}}{\text{rem}} \right) = \int \frac{\text{rem} - 1}{\text{rem}^2} \\
& \frac{\text{avl}}{\text{rem}} = \frac{1}{\text{rem}} + \ln \text{rem} + c \text{ apply initial condition } \text{avl} = \text{rem} = n \\
& 1 = \frac{1}{n} + \ln n + c \implies \\
& \text{avl} = 1 + \text{rem} \left(\ln \text{rem} - \ln n + 1 - \frac{1}{n} \right)
\end{aligned}$$

giving us the final approximation

$$\text{avl} = 1 + \text{rem} \left(\frac{n-1}{n} - \ln \frac{\text{rem}}{n} \right) \quad (4)$$

Bearing in mind that the initial Markov chain doesn't perfectly emulate the last step, we substitute $\text{rem} = 1$ instead $\text{rem} = 0$ and from Equation 4 we receive that, when only 1 online vertex has yet to arrive, the number of remaining columns is $n \frac{1}{e} + o(1)$. Bearing in mind that the last arriving vertex will affect the performance by at most 1, the expected size of the final matching is $n \left(1 - \frac{1}{e} \right)$

4 Delay Model

In the Delay model the Algorithm can wait a certain amount of time before matching a vertex

Theorem 3. *For delay no greater than $n/2$ any deterministic online algorithm has competitiveness of 0.5*

Proof. Consider the following graph G (figure above): There are $k = n/2$ groups $\{Z_i | i \in [k]\}$ of two offline vertices

First, k vertices z_i^D arrive, each connected to both vertices of Z_i .

As z_k^D arrives the cache is full, and z_1^D has to be matched. Let the offline vertex it was matched to be z_1^B . Next, z_1^C arrives, only connected to z_1^B , which is unmatchable. Let z_1^A be the unmatched offline vertex.

After z_1^A has arrived z_2^D has to be matched. The procedure above is repeated until z_k^C has arrived.

The perfect matching of G connects all z_i^A, z_i^D and z_i^B, z_i^C pairs, while the algorithm connects only z_i^B, z_i^D pairs, making the competitiveness 0.5. The final graph is illustrated in the figure below. $\square$

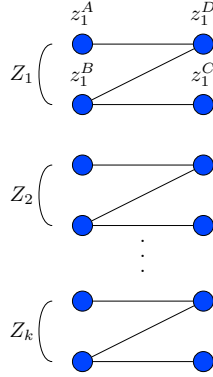


Figure 1: Worst case scenario for a deterministic cache algorithm

5 Final Degree Model

In the Final Degree Model, The algorithm has, at all times, the knowledge of the what the degree of all offline edges is in the final graph. This allows an algorithm to know how many opportunities there will be for a vertex to be matched and let it focus on the most urgent vertex. A natural expression of this idea is the effective degree algorithm.

Algorithm 4 Effective Degree

on Initialization: for each offline vertex u_i let $d_{u_i} = \deg(u_i)$

on Arrival of vertex v : N are unmatched offline neighbors of v . If $N \neq \emptyset$,
match v to $\underset{u \in N}{\operatorname{argmin}} d_u$, $\forall_{u \in N} d_u - 1$

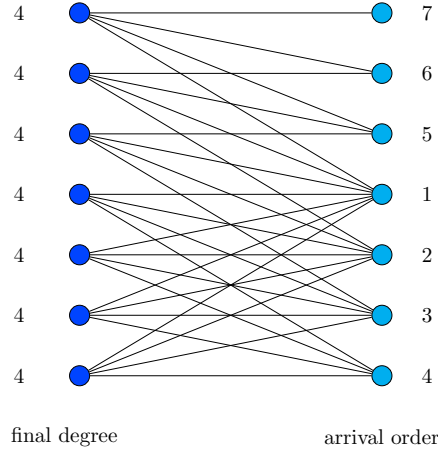
Despite this algorithm seeming promising, it is only marginally more effective than a naive deterministic approach. We will show that in this model no deterministic algorithm has asymptotic competitiveness greater than 0.5

Theorem 4. *In the Final Degree model no deterministic algorithm has a competitiveness higher than 0.5*

Proof. Consider the following graph G_{deg} which has $n = 2k + 1$ offline vertices with final degree $k + 1$

$k + 1$ vertices arrive, all connected to all unmatched offline vertices.

Notice that at this point $k + 1$ vertices are matched and the remaining k have degree $k + 1$ in the current graph, meaning that they cannot be matched. All that remains is to fulfill the degree promise. This is easy as the degrees of the matched vertices are $1, 2, \dots, k, k + 1$ meaning that all that is required is for k vertices connected to all offline vertices with unfulfilled degree promise to arrive. The size of the matching found is $k + 1$ while the graph has a perfect matching of size $n = 2k + 1$ which is achieved when the k vertices matched by the graph are instead matched to the vertices which fulfilled their degree promise while the remaining $k + 1$ vertices are matched among themselves since they form a biclique $K_{k+1, k+1}$ $\square$



We will remark here that this model is a relaxation of the Semi-Online model, so competitiveness here isn't lower

5.1 Randomized Augmentation

As shown above, the information of a graph's final degree isn't helpful for deterministic algorithms. However, there is a good reason to believe that this might not be the case for randomized algorithms

Here, the fact that the competitiveness is only 0.5 asymptotically and in actuality is $\frac{k+1}{2k+1}$ is key.

Claim 5.1. *If all vertices have different degrees then the algorithm finds a perfect matching*

Proof. Let $G_n = (V, U, E)$ be a $n \times n$ bipartite graph with all vertices in U having different degrees. WLOG G has a perfect matching, implying that the degrees of $u \in U$ are $1, \dots, n$. Let $u_k := u \in U | \deg(u) = k$

To force the algorithm to leave n_k behind, k online vertices need to arrive, each connected to a different $u'_i \in \{u_i | i < k\}$. The problem with this is that the size of U' is $k - 1$, making such a thing impossible $\square$

A much stronger realization can be gleaned from this reasoning to force a deterministic algorithm to leave a vertex u behind, and for it to be possible for a randomized algorithm to leave that vertex behind, there must be $\deg(u)$ vertices v such that $\deg(u) \geq \deg(v)$, which moreover won't be left behind.

This line of reasoning leads directly to the graph $G_{\deg}$ used in the proof of Theorem 4. However, this also leads to a problem for an adversary. That graph is rather dense, which isn't a situation where randomized graphs perform poorly. Indeed, despite RGA (Algorithm 3) having an expected competitiveness of $\frac{n}{2} + O(\log n)$, it is expected to find a matching of size $n(1 - \frac{1}{e})$ on a graph which is actually a subgraph of the degree countergraph edgewise, meaning it would work at least as well here

Algorithm 5 Randomized Effective Degree

on Initialization: for each offline vertex u_i let $d_{u_i} = \deg(u_i)$
on Arrival of vertex v : N are unmatched offline neighbors of v . If $N \neq \emptyset$, let $d = \min_{u \in N} d_u$. $v = \text{random}(\{u \in N | \deg(v) = d\})$. Match v to u , $\forall_{u \in N} d_u - 1$

Lemma 5.1. *Algorithm 5 has a performance of $jinsert_j$ on $G_{\deg}$*

Proof. Our approach here is similar to the one in subsection 3.2 for proving Theorem 2 $\square$

References

- [KVV90] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. “An optimal algorithm for on-line bipartite matching”. In: *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*. STOC '90. Baltimore, Maryland, USA: Association for Computing Machinery, 1990, pp. 352–358. ISBN: 0897913612. DOI: 10.1145/100216.100262. URL: <https://doi.org/10.1145/100216.100262>.
- [BM08] Benjamin Birnbaum and Claire Mathieu. “On-line bipartite matching made simple”. In: *SIGACT News* 39.1 (Mar. 2008), pp. 80–87. ISSN: 0163-5700. DOI: 10.1145/1360443.1360462. URL: <https://doi.org/10.1145/1360443.1360462>.